## Operating Systems

# Tutorial Worksheet 05

**Description.** This tutorial worksheet covers the topic of memory management.

1. **Concepts understanding questions**

   a) Briefly explain the difference between absolute code and relocatable code.

   b) Briefly describe the dynamic loading and dynamically linked libraries.

   c) What are the advantages of dynamically linked libraries over statically linked libraries?

   d) Explain why and how CPUs use an internal cash memory.

   e) Explain how a program is transformed from a source code (e.g., main.c) to a binary executable file (e.g., main.exe). Describe in which phase libraries and header files are linked and included.

   f) Explain the difference between internal and external fragmentation.

   g) Can we have internal fragmentation in a memory system that uses segmentation? Justify your answer.

   h) Why there is no external fragmentation in a memory system that uses paging?

   i) Briefly discuss what is the concept of virtual memory.

   j) Briefly explain what is the different between a segment fault and a page fault? Explain what happens in each event.

   k) Briefly discuss how modern operating systems that use virtual memory allow the execution of programs that have a size larger than the available RAM (Read Only Memory).

   l) Briefly describe what thrashing is. How can the operating system detect when thrashing occurs? What should the operating system do when thrashing occurs?

2. **Central memory size**

Assuming a byte-addressed memory (i.e., every byte of memory has its own address), give the size of the memory for the following address sizes:

For `M=11` the size is ... For `M=25` the size is ... For `M=36` the size is ...

For `M=44` the size is ... For `M=01` the size is ... For `M=27` the size is ...

3. **Physical and logical addresses in paging**

Consider a computer in which the virtual memory consists of 8 pages of 1024 bytes each, mapped onto physical memory of 32 page frames. Then, assuming that the memory is byte-addressed, how many bits are there in a virtual address? How many bits are there in the physical address?

4. **Single-level paging**

(a) Consider a 64-bit computer that uses single-level paging for memory management. It uses 64KB as page size and 8 bytes per page entry in the page table. Assuming a program that uses virtual addresses from 0...4GB, what is the size of the page table (a.k.a., flat page table)?

(b) Now consider the same computer from Question 2.4, what would be the size of the page table for a 4GB process, if the computer operates a 4-level paging scheme with page number split equally among the 4 levels (i.e., each level uses the same number of bits to code a page number)?

(c) Consider a 32-bit computer that has a byte-addressed memory of 1MB. It uses single-paging scheme with 4KB page size. Having the page table below, translate the following virtual addresses into physical addresses.

| **Page ♯** | **Base address** | **Validity** |
|---|---|---|
| 1 | 0x6C000 | V |
| 2 | 0xB1000 | V |
| 3 | 0xC4000 | V |
| 4 | 0xA8000 | V |
| 5 | 0xE6000 | V |
| 6 | 0xD1000 | V |
| ... | ... | V |

- Virtual address `0x00004532` has physical address:

- Virtual address `0x00006EF2` has physical address:

- Virtual address `0x00001F33` has physical address:

- Virtual address `0x000031E2` has physical address:

5. **Segmentation with paging — 1**

A computer uses segmentation with paging. A virtual address consists of six hexadecimal digits, SSPPEE, where SS is a segment number and PPEE is the offset within the segment. The segment is paged, with PP as the page number and EE as the page offset. The page size is 256 bytes. The physical address is on 16 bits. The figure below shows selected parts of the current memory contents. All numbers are hexadecimal. Segments and page tables are indexed starting at 0. Two-digit frame numbers are stored in the segment tables, in the page tables, and in STBR (Segment Table Base Register). These frame numbers are converted to main memory addresses by appending 0x00. For example, the entry 0F in a segment table means that the segment's page table is located at address 0x0F00. A * means the page is on disk.

`STBR=0x06` List of free page frames: `0x1A, 0x08, 0x21, 0x16, 0x0E`.

Segment tables:

| 0x0200 | 0x0F |
|---|---|
| | 0x13 |
| | 0x09 |
| | * |
| | 4E |

| 0x0600 | 0x09 |
|---|---|
| | 0x0A |
| | 0x4C |
| | 0x3D |
| | 0x5D |

| 0x3000 | 0x0F |
|---|---|
| | 0x09 |
| | 0x13 |
| | 0x4C |
| | * |

Page tables:

| 0x3D00 | 0x5C |
|---|---|
| | 0x1E |
| | 0x5A |
| | 59 |
| | * |

| 0x4C00 | * |
|---|---|
| | 0x3B |
| | * |
| | 0X20 |
| | * |

| 0x0A00 | * |
|---|---|
| | 0x57 |
| | * |
| | * |
| | * |

| 0x0900 | 04 |
|---|---|
| | |
| | |
| | |
| | |

| 0x0F00 | * |
|---|---|
| | 0x25 |

| 0x1300 | 0x18 |
|---|---|

| 0x4E00 | 11 |
|---|---|
| | * |

(a) The following sequence of virtual memory addresses are generated by the current process. What are the corresponding main memory addresses? In answering this question, assume that new pages are brought into main memory using the given list of free page frames (0x1A, 0x08, 0x21, 0x16, 0x0E). Write changes to the segment or page tables into the above figure.

| Virtual memory address | Main memory address | Is there a page fault? |
|---|---|---|
| 0x0000B2 | ------ | YES NO |
| 0x020216 | ------ | YES NO |
| 0x02015E | ------ | YES NO |
| 0x0202A7 | ------ | YES NO |

(b) Following the four accesses performed in part (a), the system switches execution to another process. The value `0x02` is loaded into the STBR. What segments, if any, are shared between this new process and the previous process?

(c) Describe what happens when the new process (STBR=`0x02`) references address `0x030142`. Keep using list of free page frames from part (14.1). Write changes to the segment or page tables into the above figure. Can you determine what main-memory address results after address translation? If so, state the address. If not, state what additional information is needed.

6. **Segmentation with paging — 2**

Using segmentation with paging, this table shows three methods of dividing up virtual addresses. For each case, calculate maximum size for a segment, a segment table, and a segment's page table. You may state your answers in terms of bytes, kilobytes, megabytes, gigabytes, or using a format like $2^x$ byte. Also, explain your computations.

| **Bits in virtual address** | **Page size** | **Size of entires in segment table & page table** | **Max ♯ of segments segments** | **Max size of a segment** | **Max size of a segment table** | **Max size of a segment's page table** |
|---|---|---|---|---|---|---|
| 16 bits | 128 byte | 8 byte | 64 | | | |
| 32 bits | 1024 byte | 8 byte | 4096 | | | |
| 32 bits | 8192 byte | 8 byte | 65536 | | | |

7. **Page fault rate**

Find the maximum allowable page-fault rate so that effective virtual memory access time is $\leq 200$ nanoseconds on a computer system with the following characteristics. The time to access main memory is 100 nanoseconds; this is sometimes referred to as the raw memory access time. Paging is used, with the page table stored in main memory. The address translation cache (TLB, Translation Lookaside Buffer) has a 95% hit rate. TLB access time is negligible compared to the 100 nanosecond memory access time, so:

- When address translation encounters a TLB hit, the virtual memory access time is 100 nanoseconds. (Virtual memory access time is the same as raw memory access time, because we assume that address translation time is negligible.)

- When there is a TLB miss with no page fault, the virtual memory access time is 200 nanoseconds. (Address translation must access memory to read an entry in the page table, thus adding 100 nanoseconds of overhead.) The time to handle a page fault is 8 milliseconds when a clean page is replaced, and 16 milliseconds when a dirty page is replaced. The replaced page is dirty 65% of the time; this means that 35% of the time the replaced page is clean. (A dirty page is one that has been written to since it was loaded into main memory. Replacing a dirty page takes longer because the operating system has to write the contents of the dirty page back to disk before it can reuse that page frame.)

8. **Multilevel paging**

   Consider a 32-bit computer that uses paging as a memory management scheme. Each page table entry occupies 4 bytes.

   (a) What is the smallest page size that allows the page table to fit into one page?

   (b) A page size of 1KB is chosen. This means that the page table has to be stored in multiple levels. Describe how the 32 bit virtual address is divided into fields: how many fields are there, and how many bits are in each field?

9. **Page replacement algorithms**

   The following sequence of virtual memory references is generated when a program is executed. Each memory reference is a 12 bit number written as three hexadecimal digits.

   0x019, 0x01A, 0x1E4, 0x170, 0x073, 0x30E, 0x185, 0x24B, 0x24C, 0x430, 0x458, 0x364

   (a) What is the reference string, assuming a page size of 256 Bytes?

   (b) Find the page fault rate for the reference string in part (a): assume that 2 frames of main memory are available to the program and the FIFO page replacement algorithm is used. Note that the page fault rate is calculated as "number of page faults" divided by "number of virtual memory references used to form the reference string".

   (c) Repeat (b) using the LRU (Least Recently Used) page replacement algorithm.

   (d) Repeat (b) using the optimal page replacement algorithm.

10. **Page faults in paging**

    Assume you have a reference string for a process with m frames, and initially all m frames are empty. The reference string has length p, with n distinct page numbers occurring in it. Give an upper bound and a lower bound on the number of page faults. Your bounds should hold for any page-replacement algorithm. Briefly justify your answers.

11. **Page fault and swapping**

This program swaps the elements in the first half of array A with the elements in the second half of the array:

```
var A: array[1..1000] of integer; // A is an array indexed from 1 to 1000

temp1, temp2, i:  integer;

// Code to initialize A has been omitted.

for (i := 1 to 500) // i is an index for the first half of the array.

{

    temp1 := A[i]; // This code swaps A[i] and A[500+i].

    temp2 := A[500+i];

    A[i] := temp2;

    A[500+i] := temp1;

}
```

Assume that 100 integers fit into a page, so array A occupies 10 pages. Initially all of A is on disk. How many page faults occur during program execution in cases (a) and (b)? Use Least Recently Used (LRU) page replacement. [In this problem we only consider page faults that result from accessing A: assume that there are no page faults from fetching instructions.]

(a) Five page frames of size 100 integers are allocated to array A.

(b) One page frame of size 100 integers is allocated to array A.

12. **Memory effective access time**

Consider a main memory with 220 nanoseconds raw access time, and a TLB (Translation Look-aside Buffer) cache with 120 nanoseconds access time, respectively. Knowing that the TLB hit ratio is 98%, calculate the effective access time.